

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Sree	Reg. No.:	192524023
Section / Batch:	AI and DS / 2025	Date:	30/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

1. Analyze the impact of recursion overhead in Quick Sort by measuring function call counts and depth. Record Execution time for large dataset. Interpret how recursion contributes to total execution cost. Justify optimization techniques such as tail recursion.

Code of Conduct

I Sree B(192524023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sree B

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____


```
52
53 def analyze_quick_sort(arr, use_optimized=False):
54     global call_count, max_depth
55     call_count = 0
56     max_depth = 0
57
58     arr_copy = arr.copy()
59     start_time = time.perf_counter()
60
61     if use_optimized:
62         tail_recursive_quick_sort(arr_copy, 0, len(arr_copy) - 1)
63     else:
64         quick_sort(arr_copy, 0, len(arr_copy) - 1)
65
66     end_time = time.perf_counter()
67     execution_time = end_time - start_time
68
69     return execution_time, call_count, max_depth, arr_copy
70
71
72 if __name__ == "__main__":
73     sys.setrecursionlimit(10000)
74
75     dataset_sizes = [100, 1000, 5000, 10000]
76
77     print("=" * 80)
78     print(f"{'Size':<10}{'Version':<20}{'Time (s)':<15}{'Calls':<10}{'N':<10}")
79     print("=" * 80)
80
81     for size in dataset_sizes:
82         data = [random.randint(0, 100000) for _ in range(size)]
83
84
85     time_std, calls_std, depth_std, _ = analyze_quick_sort(data, use_optimized=False)
```

```
63     else:
64         quick_sort(arr_copy, 0, len(arr_copy) - 1)
65
66     end_time = time.perf_counter()
67     execution_time = end_time - start_time
68
69     return execution_time, call_count, max_depth, arr_copy
70
71
72 if __name__ == "__main__":
73     sys.setrecursionlimit(10000)
74
75     dataset_sizes = [100, 1000, 5000, 10000]
76
77     print("=" * 80)
78     print(f"{'Size':<10}{'Version':<20}{'Time (s)':<15}{'Calls':<10}{'N':<10}")
79     print("=" * 80)
80
81     for size in dataset_sizes:
82         data = [random.randint(0, 100000) for _ in range(size)]
83
84
85         time_std, calls_std, depth_std, _ = analyze_quick_sort(data, use_optimized=False)
86         print(f"{'Size':<10}{'Standard':<20}{'time_std':<15.6f}{'calls_std':<10.6f}{'N':<10}")
87
88         time_opt, calls_opt, depth_opt, _ = analyze_quick_sort(data, use_optimized=True)
89         print(f"{'Size':<10}{'Optimized':<20}{'time_opt':<15.6f}{'calls_opt':<10.6f}{'N':<10}")
90
91     print("=" * 80)
```

Output

```
0.002065      653      6
5000      Standard
0.014530      6649     26
5000      Optimized
0.013168      3324      6
10000     Standard
```

```
100      Standard
0.000188      143      15
100      Optimized
0.000159      71       3
1000     Standard
0.002287      1307     21
```

Output

```
10000     Standard
0.032096      13349     29
10000     Optimized
0.029536      6674      7
=====
=====
```